

MEDIATEK

MT8676 Android SDK User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Dandan Hu	Official release

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

1 SDK 4

1.1 Overview 4

1.1.1 Introduction 4

1.1.2 SDK Function Introduction 4

1.1.3 Attention 4

1.2 Architecture/Process Overview 4

1.2.1 Internal Structure Diagram 4

1.2.2 API List 5

1.3 Configuration/Customization Guideline 48

1.3.1 Main Features Introduction 48

1.3.2 Special Notes on Collision Detection 49

1.4 Frequently Asked Questions/Troubleshooting 50

1.4.1 Common SDK Questions 50

1.4.2 Methods to Toggle Debug Logs 51

Exhibit 1 Terms and Conditions 52

List of Figures

Figure 1-1. SDK internal structure diagram 5

Figure 1-2. SPM SDK flow 5

1 SDK

1.1 Overview

1.1.1 Introduction

This chapter primarily introduces the MT8676 SDK interface as well as common methods for handling SDK issues.

SmartPlatform is a unique product. To reduce the difficulty of customer APK development and to meet more customized requirements, MediaTek provides a set of SDK interfaces exclusive to SmartPlatform. This enables developers to quickly and conveniently develop quality applications.

1.1.2 SDK Function Introduction

The SDK provides APIs for customers to call, allowing each camera to independently perform preview, capture, record, and YUV callback (PictureSequence). Additionally, it supports multiple processes using a single camera simultaneously.

1.1.3 Attention

- In the SDK, considering synchronization and efficiency issues, callbacks are processed directly in the binder thread without creating a new thread. Therefore, it is advisable for the app to start a new thread to handle the callback after receiving it. If a new thread is not started, do not continue to call SDK interfaces within the callback (as it may cause a deadlock); also, avoid performing time-consuming tasks within the callback.
- Regarding the photography process, to avoid extending the duration of taking photos and interrupting the preview/record, the current approach binds the photography and preview processes together. If you need to adjust the size of the photo, it should be set through `setPictureSize` before setting the `setPreviewSurface`.

1.2 Architecture/Process Overview

1.2.1 Internal Structure Diagram

The internal structure diagram of the MT8676 SDK is shown in [Figure 1-1](#):

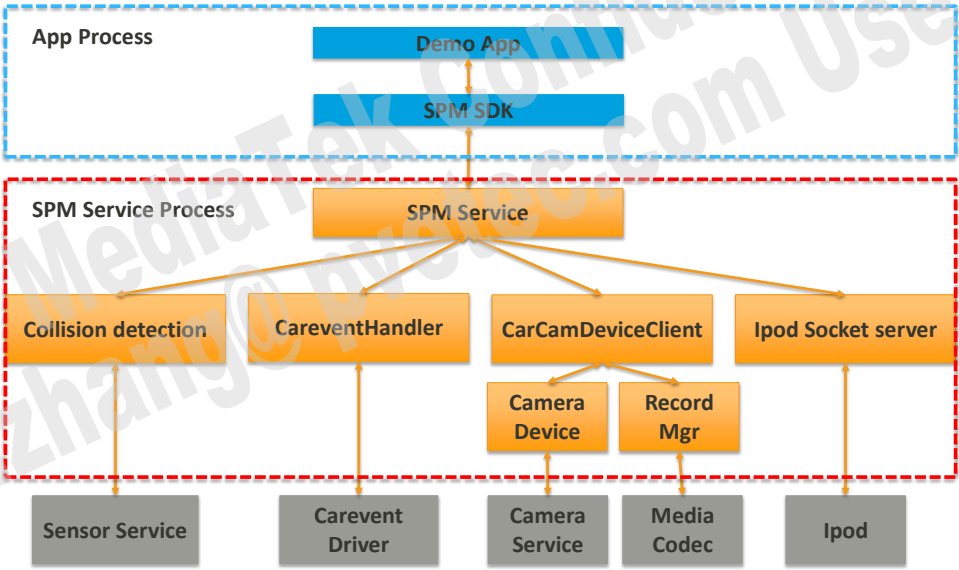


Figure 1-1. SDK internal structure diagram

MT8676 SDK flow is shown in Figure 1-2:

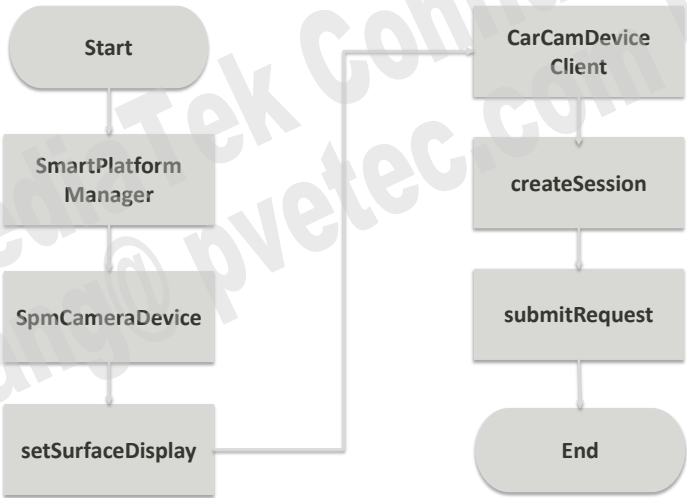


Figure 1-2. SPM SDK flow

1.2.2 API List

1.2.2.1 SmartPlatformManager

Description:

The primary interface class of the SmartPlatform SDK, which is a singleton pattern. It manages the platform’s camera recording, collision detection, shutdown state settings, etc.

Import SmartPlatformManager:

```
import com.mediatek.smartplatform.SmartPlatformManager;
```

Instantiate (create) an object:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
```

1.2.2.1.1 get()

Function:

```
public static SmartPlatformManager get()
```

Returns:

Returns an instance of SmartPlatformManager.

Details:

SmartPlatformManager exists in the app as a singleton, which means that the same instance of SmartPlatformManager is returned wherever this method is called within the same app. The SmartPlatformManager instance is created when the class is loaded, following the “eager initialization” pattern.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
```

1.2.2.1.2 isServiceAlive

Function:

```
public boolean isServiceAlive()
```

Returns:

Indicates whether the SmartPlatform service and Camera Service obtained by the APK process are available.

Details:

- If the SmartPlatform Service or Camera Service has not been registered with the system services, or if the registration fails, this interface will return false. This means that some calling methods on the SmartPlatformManager object will result in exceptions. Therefore, the app needs to ensure that the SmartPlatform service is available before calling related interfaces.
- This interface typically returns false when the smartplatformserver process exits unexpectedly. The smartplatformserver process will restart, and at this time, the app will also restart. However, the app may restart before the smartplatformserver process, and at this moment, the SmartPlatform service may not be ready, leading to a NullPointerException when the app calls related interfaces, causing the app to exit.
- For the above situation, it is recommended that the following logic operations should be implemented for the app:
 1. When the app restarts after an unexpected exit, first determine if the service is available (only needs to be checked once).
 2. If not available, delay and wait for the service to be ready. Do not call related interfaces (such as openCameraDevice()) before the service is ready.
 3. Once the service is ready, proceed with the normal process.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
if (!mSmartPlatformManager.isServiceAlive()){
    // You can delay and wait before determining if it is available
    mHandler.sendEmptyMessageDelay(INIT_DELAY, 500);
} else{
    // Follow the normal initialization process.
}
```

1.2.2.1.3 getCameraIdList

Function:

```
public String[] getCameraIdList ()
```

Returns:

Returns a list of camera devices currently connected to the platform, including those in use.

Details:

The camera list obtained by this function refers to the cameras that are currently physically connected to the platform.

Unlike `getCameraIdList` in `android.hardware.camera2.CameraManager`, this function returns cameras that are present (1), not present (0), and in use (-2). The `CameraManager` only returns cameras that are present. For specific status information, refer to `getCameraStatus`.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
String cameraList[] = mSmartPlatformManager.getCameraIdList();
```

1.2.2.1.4 getAvmCameraId

Function:

```
public String getAvmCameraId ()
```

Returns:

Returns the ID of the Around View Monitor (AVM) camera device currently connected to the platform.

Details:

The camera list obtained by this function refers to the AVM camera IDs that are currently physically connected to the platform. These camera IDs are mainly used for the CarcorderDemo APK calls and are suitable for AVM DVR.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
String cameraId = mSmartPlatformManager.getAvmCameraId();
```

1.2.2.1.5 getCameraCharacteristics

Function:

```
public CameraCharacteristics getCameraCharacteristics(String cameraId)
```

Parameters:

`cameraId` The identifier corresponding to the camera

Returns:

The data type is `android.hardware.camera2.CameraCharacteristics`. The `cameraId` corresponds to the device capabilities of the camera.

Details:

The `cameraId` is obtained through `getCameraIdList`. This API is equivalent to `getCameraCharacteristics` in `android.hardware.camera2.CameraManager`.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
String cameraList[] = mSmartPlatformManager.getCameraIdList();
```

```
mSmartPlatformManager.getCameraCharacteristics(cameraList[0]);
```

1.2.2.1.6 openCameraDevice

Function:

```
public SpmCameraDevice openCameraDevice(String cameraIdStr)
```

Parameters:

cameraIdStr The identifier corresponding to the camera.

Returns:

Returns a SpmCameraDevice object.

Details:

- The parameter cameraIdStr is the identifier corresponding to the camera, with each camera having a unique ID value. This ID is obtained through getCameraIdList.
- Within the same app, calling this function with the same ID each time will result in the same CameraDevice object. The object is only created upon the first call.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
String ids[] = mSmartPlatformManager.getCameraIdList();  
SpmCameraDevice cameraDevice0= mSmartPlatformManager.openCameraDevice(ids[0]);
```

1.2.2.1.7 openAvmCameraDevice

Function:

```
public SpmCameraDevice openCameraDevice()
```

Returns:

Returns a SpmCameraDevice object.

Details:

Within the same app, each call to this function will result in the same CameraDevice object. The object is only created upon the first call.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
SpmCameraDevice avmcameraDevice= mSmartPlatformManager.openAvmCameraDevice();
```

1.2.2.1.8 closeCameraDevice

Function:

```
public void closeCameraDevice(String cameraId)
```

Parameters:

cameraId The identifier corresponding to the camera

Details:

- The parameter cameraId is the identifier ID corresponding to the camera, with each camera having a unique ID value. This ID is obtained through getCameraIdList. If this CameraDevice is no longer needed, you should call this function to shut it down, allowing for the release and recycling of resources.

- After calling this function, all references previously obtained through `openCameraDevice` will no longer be available.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
String ids[] = mSmartPlatformManager.getCameraIdList();
SpmCameraDevice cameraDevice0= mSmartPlatformManager.openCameraDevice(ids[0]);
mSmartPlatformManager.closeCameraDevice(ids[0]);
```

1.2.2.1.9 closeAvmCameraDevice

Function:

```
public void closeAvmCameraDevice()
```

Details:

After calling this function, the references previously obtained through `openAvmCameraDevice` will no longer be available.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
SpmCameraDevice avmcameraDevice0= mSmartPlatformManager.openAvmCameraDevice();
mSmartPlatformManager.closeAvmCameraDevice();
```

1.2.2.1.10 addServiceDeathCallback

Function:

```
public void addServiceDeathCallback(ServiceDeathCallback callback)
```

Parameters:

Callback for when the SmartPlatformServer process exits (dies).

Details:

- `ServiceDeathCallback` is defined as follows:

```
public interface ServiceDeathCallback {
    void onDeath(int arg1,String arg2);
}
```

- If the same `ServiceDeathCallback` is added multiple times, there will only be one instance of the callback.
- The triggering of this callback means that the SmartPlatformServer process has died, and at this point, the related services of SmartPlatform Service are no longer available. The current mechanism is that after the SmartPlatformServer dies (is killed unexpectedly), it will be reloaded and run again. The app needs to re-acquire the service, as the previous SmartPlatformManager, SpmCameraDevice objects can no longer be used.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
mSmartPlatformManager.addServiceDeathCallback (
    new ServiceDeathCallback() {
        public void onDeath(int arg1,String arg2){
        }
    }
);
```

1.2.2.1.11 removeServiceDeathCallback

Function:

```
public void removeServiceDeathCallback (ServiceDeathCallback callback)
```

Parameters:

callback The callback when the SmartPlatformServer process exits (dies)

Details:

Removes the registered callback object.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
mSmartPlatformManager.removeServiceDeathCallback(callback);
```

1.2.2.1.12 addCameraAvailableCallback

Function:

```
public void addCameraAvailableCallback(ServiceDeathCallback callback)
```

Parameters:

callback The callback that returns the camera status

Details:

- CameraAvailableCallback is defined as follows:

```
public interface CameraAvailableCallback {
    void onAvailable (String cameraId, int status);
}
```

- If the same CameraAvailableCallback is added multiple times, there will only be one instance of the callback.
- The triggering of this callback means that a camera has been added or removed. When a camera is removed, you can perform operations related to stopping the recording.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
mSmartPlatformManager.addCameraAvailableCallback (
    new CameraAvailableCallback () {
        public void onAvailable (String cameraId, int status){
        }
    }
);
```

1.2.2.1.13 removeCameraAvailableCallback

Function:

```
public void removeCameraAvailableCallback (CameraAvailableCallback callback)
```

Parameters:

callback The callback that returns the camera status

Details:

Removes the registered callback object.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.removeCameraAvailableCallback(callback);
```

1.2.2.1.14 getIpodProxy

Function:

```
public IpodProxy getIpodProxy()
```

Returns:

Returns a IpodProxy object.

Details:

Refer to the IpodProxy introduction.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
IpodProxy mIpodProxy=mSmartPlatformManager.getIpodProxy();
```

1.2.2.1.15 getCollisionProxy

Function:

```
public CollisionProxy getCollisionProxy()
```

Returns:

Returns a CollisionProxy object.

Details:

Refer to the CollisionProxy introduction.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
CollisionProxy mCollisionProxy=mSmartPlatformManager.getCollisionProxy();
```

1.2.2.1.16 getCarEventProxy

Function:

```
public CarEventProxy getCarEventProxy()
```

Returns:

Returns a CarEventProxy object.

Details:

Refer to the CarEventProxy introduction.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
CarEventProxy mCarEventProxy=mSmartPlatformManager.getCarEventProxy();
```

1.2.2.1.17 getNumberOfCameras

Function:

```
public int getNumberOfCamera()
```

Returns:

Returns the number of available cameras, including those that are present, not present, and in use.

Details:

Refer to the description of `getCameraIdList`; essentially, this method returns the size of the array returned by `getCameraIdList`.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
int cameraNumber = mSmartPlatformManager.getNumberOfCamera();
```

1.2.2.1.18 getCameraStatus

Function:

```
public int getCameraStatus(String cameraId)
```

Returns:

Returns the status of the camera ID being requested.

Details:

The meanings of the statuses are as described in

`frameworks/av/camera/aidl/android/hardware/ICameraServiceListener.aidl`:

```
// Device physically unplugged
const int STATUS_NOT_PRESENT = 0; // Not present
// Device physically has been plugged in and the camera can be used exclusively
const int STATUS_PRESENT = 1; // Present and available
// Device physically has been plugged in but it will not be connect-able until enumeration
is complete
const int STATUS_ENUMERATING = 2; // Status returned by the camera when the enumerator
provider is running, will change to PRESENT after enumeration is complete
// Camera is in use by another app and cannot be used exclusively
const int STATUS_NOT_AVAILABLE = -2; // In use by another process/app, but since MediaTek
SDK supports multi-process usage, it is considered available
// Use to initialize variables only
const int STATUS_UNKNOWN = -1; // Initial state
```

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
int cameraStatus = mSmartPlatformManager.getCameraStatus(cameraId);
```

1.2.2.2 PreviewSource

Import PreviewSource:

```
import com.mediatek.smartplatform.PreviewSource;
```

Object Description:

Used to distinguish between multiple previews. The same camera device can have individual previews. Typically, `GENERAL_CAMERA` or `PREVIEW_[0-4]` is used.

1.2.2.3 RecordSource

Import RecordSource:

```
import com.mediatek.smartplatform.RecordSource;
```

Object Description:

Used to distinguish the type of recording. The same camera device can support multiple streams of recording at different aspect ratios. Typically, `GENERAL_CAMERA` or `RECORD_[0-4]` is used for recording. For low-resolution recording, `GENERAL_CAMERA_SUB` or `RECORD_EX_[0-4]` can be selected.

1.2.2.4 PictureSequenceSource

Import PictureSequenceSource:

```
import com.mediatek.smartplatform.PictureSequenceSource;
```

Object Description:

Used to distinguish the type of `PictureSequenceSource`. The same camera device can capture multiple streams of continuous images at different resolutions. Typically, `GENERAL_CAMERA` is used.

1.2.2.5 RecordConfiguration

Import RecordConfiguration:

```
import com.mediatek.smartplatform.RecordConfiguration;
```

Instantiate (create) an object:

```
RecordConfiguration recordConfig= RecordConfiguration.get(recordSource);
```

1.2.2.5.1 mCamcorderProfile

Class: `android.media.CamcorderProfile`

Detail: Settings for the video file format, frame rate, and video resolution.

1.2.2.5.2 mOutputFilePath

Class: `java.lang.String`

Detail: The save path for the record video file.

1.2.2.5.3 mOutputFileName

Class: `java.lang.String`

Detail: The prefix for the record video file.

1.2.2.5.4 mVideoCallback

Class: com.mediatek.smartplatform.SpmCameraDevice.VideoCallback

Detail: The callback when video-related events occur.

- VideoCallback is defined as follows:

```
public interface VideoCallback {
    public static final int VIDEO_EVNET_ADD_FILE_IN_GALLERY=0;
    public static final int VIDEO_EVNET_DELETE_FILE_IN_GALLERY=1;
    public static final int VIDEO_EVNET_SDCARD_FULL=2;
    public static final int VIDEO_EVNET_RECORD_STATUS_START=3;
    public static final int VIDEO_EVNET_RECORD_STATUS_STOP =4;
    public static final int VIDEO_EVNET_RECORD_RECORDING_ERROR=5;
    public static final int VIDEO_EVNET_RECORD_SDCARD_DAMAGED=6;
    public static final int VIDEO_EVNET_LOWRES_KEYPOINT_START =7;
    public static final int VIDEO_EVNET_LOWRES_KEYPOINT_STOP =8;
    public static final int VIDEO_EVNET_KEYPOINT_START =9;
    public static final int VIDEO_EVNET_KEYPOINT_STOP =10;

    void onVideoTaken(int eventType,int cameraId, int recorderSource, String videoname,int
    startTime, int endTime);
    void onVideoTake(VideoInfo videoInfo);
    void onVideoFrame(byte[] data,int dataType,int size,int cameraId, int recorderSource);
}
```

- VideoInfo is a collection of eventType, cameraId, etc., and the use of multiple parameters in onVideoTaken will be discontinued in the future.
- The meanings of eventType in onVideoTaken with [VideoInfo] are as follows:
 - VIDEO_EVNET_ADD_FILE_IN_GALLERY Indicates that the recording is complete, and the video file has been successfully saved. The video file path can be obtained through videoname.
 - VIDEO_EVNET_DELETE_FILE_IN_GALLERY Received when a video file is deleted, which generally occurs during loop recording to free up space by deleting previous video files.
 - VIDEO_EVNET_SDCARD_FULL Detected when the SD Card is full during video recording.
 - VIDEO_EVNET_RECORD_RECORDING_ERROR Occurs when there is an error during video recording.
 - VIDEO_EVNET_RECORD_SDCARD_DAMAGED Detected when the SD Card is damaged during video recording. If the write speed of the SD Card is below a certain threshold, it may also be considered damaged.
 - VIDEO_EVNET_KEYPOINT_START Indicates the start of keypoint marking.
 - VIDEO_EVNET_KEYPOINT_STOP Indicates the end of keypoint marking.
- The startTime and endTime in onVideoTaken with [VideoInfo]:

When VIDEO_EVENT_ADD_FILE_IN_GALLERY is received, it indicates the start and end time of the video file recording. Note that enableVideoWithTimeCallback(true) must be set.
- onVideoFrame allows receiving each frame of video data:

data: The buffer to save the data.

dataType: A reserved parameter that may be used to distinguish between HD and SD data.

size: Indicates the size of the video data, which is generally less than the size of the data array. The size of the video data obtained in each callback may vary..

cameraId: Returns the cameraId of the current video data.

recorderType: Returns the recorderType of the current video data
- recorderSource corresponds to the recorderSource in startRecord.

1.2.2.5.5 mAudioCallback

Class: com.mediatek.smartplatform.SpmCameraDevice.AudioCallback

Detail: The callback for audio data

- AudioCallback is defined as follows:

```
public interface AudioCallback {
    void onAudioFrame(byte[] data, int dataType, int size, int cameraId, int recorderSource);
}
```

- onAudioFrame allows receiving each frame of audio data:
 - data: The buffer to save the data.
 - dataType: A reserved parameter.
 - size: The size of the valid data, which is generally less than the size of data.
 - cameraId: The cameraId currently returning the audio data
- recorderSource: The recorder source currently returning the audio data, corresponding to the one in startRecord.

1.2.2.5.6 mRecordStatusCallback

Class: com.mediatek.smartplatform.SpmCameraDevice.RecordStatusCallback

Detail: The callback when the recording status changes

Details:

- RecordStatusCallbac is defined as follows:

```
public interface RecordStatusCallback{
    public static final int RECORD_STATUS_START      = 0;
    public static final int RECORD_STATUS_STOP       = 1;
    public static final int RECORD_STATUS_RECORDING  = 2;
    void onRecordStatusChanged(int status,int cameraid,int recorderSource);
}
```

- The callback is triggered when the recording status changes.

1.2.2.5.7 mKeypointCallback

Class: com.mediatek.smartplatform.SpmCameraDevice.KeypointCallback

Detail: The callback when keypoint data is notified to the app

- KeypointCallback is defined as follows:

```
public interface KeypointCallback{
    void onKeypointFrame(byte[] data,int dataType,int size,String cameraId, int recorderSource);
}
```

- onKeypointFrame allows receiving the encoded data:
 - data: The buffer to save the data. The same buffer is used each time the onKeypointFrame is called back. If you want to save the data, you need to copy it elsewhere; otherwise, it will be overwritten the next time.
 - dataType: A reserved parameter that may be used to distinguish between HD and SD data.

size: Indicates the size of the keypoint data, which is generally less than the size of data. The data size is typically 128KB and includes both video and audio data in the Transport Stream (TS) format.

1.2.2.5.8 mVideoFrameMode

Class: `int`

Detail: Indicates how to handle the video data.

Definition of VideoFrameMode:

- `VIDEO_FRAME_MODE_DISABLE` (0), // Record to file.
- `VIDEO_FRAME_MODE_SOURCE` (1), //Provide to the app in H264 format.
- `VIDEO_FRAME_MODE_PACKET` (2), //Provide to the app in TS (Transport Stream) format.
- `VIDEO_FRAME_MODE_DUAL` (3); //Record to file in TS format, while also providing to the app.

1.2.2.5.9 mAudioSource

Class: `int`

Detail: Indicates the source of the audio. Refer to `android.media.MediaRecorder.AudioSource`, typically set to `CAMCORDER`.

1.2.2.5.10 mKeypointSpanLimit

Class: `int`

Detail: The maximum time span, in seconds.

- If the time of the current keypoint video file and the time of the previous video file exceed the set maximum time span, the previous video file will no longer be clipped as a keypoint protection video.
- This parameter needs to be set before `startRecord`. It is an integer value and the unit is seconds.
- If the app does not set this parameter, the default span time will be the length of the currently set recording file time.

1.2.2.5.11 mRecordingMuteAudio

Class: `boolean`

Detail: Whether to turn off the microphone. Setting the parameter to true turns off the microphone, resulting in a video without sound. Setting it to false turns it on.

1.2.2.5.12 mVideoCbFrameRate

Class: `int`

Detail: Set the frame rate for video callback.

1.2.2.5.13 mLockFilePath

Class: String

Detail: The save directory for protected video files. Set the save directory for protected video files. The default path is /sdcard/DCIM/camera/protect.

1.2.2.5.14 mVideoRotateSize

Class: int

Detail: The maximum size of the recording file, calculated in bytes. When the recording file reaches this size, it will be stored in a new file.

1.2.2.5.15 mVideoRotateDuration

Class: int

Detail: Set the duration of each saved video file, measured in milliseconds.

1.2.2.5.16 mEnableRecordStartRing

Class: boolean

Detail: Toggle recording ringtone.

1.2.2.5.17 mVideoBitRateMin

Class: int

Detail: Minimum bitrate for video recording. Set the range of bitrate for video files. During recording, if the speed of writing to the file slows down, the underlying system will reduce the bitrate. If the writing speed is fast, the bitrate will be increased. Reducing the bitrate is to avoid buffer backlog caused by slow file writing. Even if the file writing speed is very slow, the adjusted bitrate will not fall below the set minimum value, and similarly, the increased bitrate will not exceed the maximum value.

1.2.2.5.18 mVideoBitRateMax

Class: int

Detail: Maximum bitrate for video recording. Set the range of bitrate for video files. During recording, if the speed of writing to the file slows down, the underlying system will reduce the bitrate. If the writing speed is fast, the bitrate will be increased. The purpose of reducing the bitrate is to prevent buffer backlog due to slow file writing. Even if the file writing speed becomes very slow, the adjusted bitrate will not be less than the set minimum value, and similarly, the increased bitrate will not exceed the set maximum value.

1.2.2.5.19 mVideoCycleDeleteFileNum

Class: int

Detail: Number of files to delete in a loop when storage space is insufficient.

1.2.2.5.20 mReduceRecordingFps

Class: int

Detail: Reduce the frame rate percentage for video recording.

FULL: 100%

HALF: 50%

THIRTY_PERCENT: 30%

QUARTER: 25%

1.2.2.6 PictureConfiguration

Import PictureConfiguration:

```
import com.mediatek.smartplatform.PictureConfiguration;
```

Instantiate (create) an object:

```
PictureConfiguration recordConfig= PictureConfiguration.get(recordSource);
```

1.2.2.6.1 mImageCallback

It is recommended to directly use ImageDataCallback.

Class: com.mediatek.smartplatform.ImageReaderEx.ImageCallback;

Detail: The callback for obtaining the YUV data of the image.

```
ImageCallback {
int IMAGE_FORMAT_YUV_420_888 = ImageFormat.YUV_420_888;
int IMAGE_FORMAT_JPEG =ImageFormat.JPEG
int IMAGE_FORMAT_NV21 =ImageFormat.NV21
int IMAGE_DATA_RAW =0;
int IMAGE_DATE_FILE =1;
void onImageAvailable(String cameraId, int format, int status , byte[] data, String path)
}
```

imageFormat: Currently, only YUV_420_888, JPEG, and NV21 are supported. The DataType can be either file or raw. If set to raw, it will be returned through a callback; if set to file, it will be saved in the specified path directory. Saving as a file is generally used for testing purposes.

1.2.2.6.2 mImageDataCallback

Class: com.mediatek.smartplatform.SpmCameraDevice.ImageDataCallback;

Detail: The callback for obtaining the YUV DirectByteBuffer of the image.

```

ImageDataCallback {
    int IMAGE_FORMAT_YUV_420_888 = ImageFormat.YUV_420_888;
    int IMAGE_FORMAT_JPEG = ImageFormat.JPEG;
    int IMAGE_FORMAT_NV21 = ImageFormat.NV21;
    int IMAGE_FORMAT_YV12 = ImageFormat.YV12;
    int IMAGE_FORMAT_RGB_888 = PixelFormat.RGB_888;
    int IMAGE_FORMAT_YUY2 = ImageFormat.YUY2;

    int IMAGE_DATA_RAW = 0;
    int IMAGE_DATA_FILE = 1;
    int IMAGE_DATA_BUFFER = 2;
    int IMAGE_DATA_IMAGE = 3;
    void onImageAvailable(ImageDataCallbackInfo imageDataCallbackInfo);
}

```

imageFormat: Currently supported formats include YUV_420_888, JPEG, NV21, YV12, RGB_888, and YUY2. The DataType can be RAW, FILE, BUFFER, or IMAGE.

RAW: The data is placed in a byte array and returned via callback.

FILE: The data is saved on the platform, old files will be deleted, used for local testing.

BUFFER: The data is placed in a DirectByteBuffer and returned to the app, reducing the number of copies.

IMAGE: The data is placed in an image and returned to the app.

1.2.2.6.3 mPath

Class: String

Detail: The save path for the image to be obtained.

This parameter is currently not in use.

1.2.2.6.4 mImageFormat

Class: int

Detail: The format of the image that the registered callback needs to obtain. The value should be set to the format value found in ImageCallback or mImageDataCallback, indicating the type of image that the callback will retrieve.

1.2.2.6.5 mDataType

Class: int

Detail: The data type of the image to be obtained, which is one of RAW, FILE, BUFFER, or IMAGE.

1.2.2.6.6 mPicWidth

Class: int

Detail: The width of the image to be obtained.

1.2.2.6.7 mPicHeight

Class: int

Detail: The height of the image to be obtained.

1.2.2.6.8 mJpegHwEnc

Class: Boolean

Detail: Whether to use a hardware encoder to encode the camera data into JPEG format; **this flag is currently not in use.**

1.2.2.7 SpmCameraDevice

Import SpmCameraDevice:

```
import com.mediatek.smartplatform.SpmCameraDevice;
```

Instantiate (create) an object:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
```

1.2.2.7.1 getState

Function:

```
public int getState()
```

Returns:

Returns the current state of the camera (CameraDevice).

Details:

- States of the Camera:
STATE_IDLE, STATE_PREVIEW_[0-4], STATE_RECORD_[0-4], etc.
- Explanation of the states:
The various states are combined using a bitwise OR. For example, if the return value is 0x21, it indicates that the CameraDevice is currently in both STATE_PREVIEW_0 and STATE_RECORD_0.
- This interface returns the state saved in the lower layer. When App1 is in preview and App2 is recording; if App1 exits unexpectedly, after App1 restarts, the current camera status obtained through this interface is still "recording".

Examples:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice(ids[0]);  
int state=mCameraDevice.getState();
```

1.2.2.7.2 getCameraCharacteristics

Function:

```
public CameraCharacteristics getCameraCharacteristics()
```

Returns:

The data type is `android.hardware.camera2.CameraCharacteristics`. It represents the device capabilities of the current camera.

Details:

The `cameraId` is obtained through `getCameraIdList`. Similar to `getCameraCharacteristics` in `android.hardware.camera2.CameraManager` and `getCameraCharacteristics` in `SmartPlatformManager`. Through this method, all configurations of the Camera can be accessed. The parameters obtained in `getParameters` are also converted through `CameraCharacteristics`.

Examples:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
mCameraDevice.getCameraCharacteristics();
```

1.2.2.7.3 getParameters

Function:

```
public SpmParameters getParameters()
```

Returns:

`SpmParameters` object, a collection of `SpmCameraDevice` parameters.

Details:

The parameter information of `SpmCameraDevice` can be obtained and modified through `SpmParameters`. The parameters in `SpmParameters` are converted from `CameraCharacteristics`. Currently, there may be some parameters that have not been converted; additional conversions can be added if needed.

Examples:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
SpmParameters params=mCameraDevice.getParameters();
```

1.2.2.7.4 setParameters

Function:

```
public void setParameters(SpmParameters params)
```

Parameters:

`params` The collection of parameters that needs to be set

Details:

When you need to modify some parameters of `SpmCameraDevice`, you must call this interface for the changes to take effect.

Examples:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
SpmParameters params=mCameraDevice.getParameters(); //First, get the params
mCameraDevice.setParameters(params); //After modifications, set them back down
```

1.2.2.7.5 setADASCallback

Function:

```
public void setADASCallback(ADASCallback callback)
```

Parameters:

callback The callback for ADAS

Details:

ADASCallback is defined as follows:

```
public interface ADASCallback{
    void onADASCallback(ADASInfo info);
}
```

Examples:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
mCameraDevice.setADASCallback(
    new setADASCallback(){
        public void onADASCallback(ADASInfo info){
            //TODO
        }
    }
);
```

1.2.2.7.6 setPictureSize

Function:

```
public int setPictureSize(android.util.Size pictureSize)
```

Parameters:

pictureSize

Returns:

Returning 0 indicates the setting was successful.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.setPictureSize (1280*720);
```

1.2.2.7.7 setPreviewSurface

Function:

```
public int setPreviewSurface(Surface surface, @PreviewSource.Format int previewSource)
```

Parameters:

surface The producer side of the display buffer

previewSource The source of the preview

Returns:

Returns a boolean indicating whether the setting was successful.

Details:

Refer to frameworks\base\core\java\android\view\Surface.java.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
SurfaceView surfaceView=findViewById(R.id.surface_view);
SurfaceHolder holder=surfaceView.getHolder();
mDevice.setPreviewSurface(holder.getSurface(), PreviewSource.GENERAL_CAMERA);
```

1.2.2.7.8 startPreview

Function:

```
public void startPreview()
```

Details:

Display the camera preview screen. Before calling this API, setPreviewSurface() must be called first.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
SurfaceView surfaceView=findViewById(R.id.surface_view);
SurfaceHolder holder=surfaceView.getHolder();
mDevice.setPreviewSurface(holder.getSurface(), PreviewSource.GENERAL_CAMERA);
mDevice.startPreview();
```

1.2.2.7.9 stopPreview

Function:

```
public void stopPreview()
```

Details:

Stop displaying the camera preview screen. After stopping the preview, you need to call setPreviewSurface(null).

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.stopPreview();
```

1.2.2.7.10 startADAS

Function:

```
public void startADAS()
```

Details:

1. Turn on the ADAS feature.
2. setADASCallback is required to receive related ADAS data.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.startADAS();
```

1.2.2.7.11 stopADAS

Function:

```
public void stopADAS()
```

Details:

Turn off the ADAS feature.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.stopADAS();
```

1.2.2.7.12 startRecord

Function:

```
public int startRecord(@RecordSource.Format int recordSource, RecordConfiguration recordConfig)
```

Parameters:

recordSource The source of the recording

recordConfig The configuration for the recording

Returns:

Returns an integer value indicating whether the recording was successfully started; 0 indicates success, otherwise failure.

Details:

For the parameter settings, refer to [RecordConfiguration](#).

Examples:

```
RecordConfiguration recordConfig= RecordConfiguration.get(recordSource);  
recordConfig.setVideoCallback(mVideoCallback);  
mCameraDevice.startRecord(RecordSource.GENERAL_CAMERA, recordConfig);
```

1.2.2.7.13 stopRecord

Function:

```
public void stopRecord(@RecordSource.Format int recordSource)
```

Parameters:

recordSource The source of the recording

Details:

Stop recording video, corresponding to startRecord().

Examples:

```
SpmCameraDevice Device = SmartPlatformManager.get().openCameraDevice("0");  
mDevice.stopRecord();
```

1.2.2.7.14 lockRecordingVideo

Function:

```
public void lockRecordingVideo(int duration,String protectedType, @RecordSource.Format int recordSource)
```

Parameters:

duration The duration of the keypoint protection

protectedType The type of keypoint protection, can be an empty string "" or "LowRes"

recordSource The source of the recording

Details:

When setting the duration for keypoint protection, the system will protect video files of length equal to the duration before and after the current timestamp. Therefore, the total length of the protected video files will be 2 * duration. The second parameter must be "LowRes", the underlying system will convert the keypoint video files to 480P and notify the app for use. The app needs to use KeypointCallback to receive the data.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
```

```
mDevice.lockRecordingVideo(20,"LowRes",RecordSource.GENERAL_CAMERA);
```

1.2.2.7.15 unlockRecordingVideo

Function:

```
public void unlockRecordingVideo(String filename, @RecordSource.Format int recordSource)
```

Parameters:

filename The path of the protected file

recordSource The source of the recording

Details:

Move a video file from a protected (protect) directory to a regular directory.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.unlockRecordingVideo(videofile, RecordSource.GENERAL_CAMERA);
```

1.2.2.7.16 setVideoRotateDuration

Function:

```
public void setVideoRotateDuration(int duration_ms, @RecordSource.Format int recordSource)
```

Parameters:

duration_ms The duration of each video file, in milliseconds

recordSource The source of the recording

Details:

Set the duration of each saved video file. The duration needs to be set after starting the recording. If it needs to be set before starting the recording, the setting can be achieved through RecordConfiguration's setVideoRotateDuration. The parameters set will take effect in the next recorded video file.

Examples:

```
CameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.setVideoRotateDuration(60*1000, RecordSource.GENERAL_CAMERA);
```

1.2.2.7.17 setVideoBitrateDyn

Function:

```
public void setVideoBitrateDyn(int bitrate,int bAdjust, @RecordSource.Format int recordSource)
```

Parameters:

bitrate The video bitrate value

bAdjust A reserved parameter, currently not in use

recordSource The source of the recording

Details:

Dynamically adjust the recording bitrate, which can be set during the recording process.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
```

```
mCameraDevice.setVideoBitrateDyn(6*1000*1000, 0, RecordSource.GENERAL_CAMERA);
```

1.2.2.7.18 setRecordingMuteAudio

Function:

```
public void setRecordingMuteAudio(boolean isMuteAudio, @RecordSource.Format int recordSource)
```

Parameters:

isMuteAudio Indicates whether to turn off the microphone.

recordSource The source of the recording

Details:

Setting the parameter to true turns off the microphone, resulting in a video without sound. Setting it to false turns the microphone on.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");
mDevice.setRecordingMuteAudio(false, RecordSource.GENERAL_CAMERA);
```

1.2.2.7.19 setProtectRecording

Function:

```
public void setProtectRecording(boolean ismotiondetect, boolean isrecordingstatus, int duration_ms, @RecordSource.Format int recordSource)
```

Parameters:

ismotiondetect Indicates whether to enable collision protection.

isrecordingstatus Specifies if the collision occurs during recording or in a non-recording state.

duration_ms The duration of collision protection

recordSource The source of the recording

Details:

This interface is used to start and stop collision protection. The first parameter indicates whether to start or stop.

Examples:

```
CameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice(0);
mCameraDevice.setProtectRecording(true, true, 5*60*1000, RecordSource.GENERAL_CAMERA)
```

1.2.2.7.20 flushCurRecFile

Function:

```
public void flushCurRecFile(@RecordSource.Format int recordSource)
```

Parameters:

recordSource The source of the recording

Details:

Flush the current recording file, writing cached data to disk.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");
```

```
mDevice. flushCurRecFile (RecordSource.GENERAL_CAMERA);
```

1.2.2.7.21 setLockFilePath

Function:

```
public void setLockFilePath(String path, @RecordSource.Format int recordSource)
```

Parameters:

path The directory where the video files are saved

recordSource The source of the recording

Details:

Set the directory for keypoint video files. The default path is /sdcard/DCIM/camera/protect/. This interface can be called before marking keypoints during the recording process.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");  
mDevice. setLockFilePath ("/sdcard/DCIM/camera/video", RecordSource.GENERAL_CAMERA);
```

1.2.2.7.22 setRecordingSdcardPath

Function:

```
public void setRecordingSdcardPath(String path, @RecordSource.Format int recordSource)
```

Parameters:

path The directory where the video files are saved

recordSource The source of the recording

Details:

The directory for saving video files, with the default path being /sdcard/DCIM/camera. This interface can be called during the recording process.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");  
mDevice.setRecordingSdcardPath("/sdcard/DCIM/camera/video",  
RecordSource.GENERAL_CAMERA);
```

1.2.2.7.23 enableShutterSound

Function:

```
public void enableShutterSound(boolean shutter)
```

Parameters:

shutter Indicates whether to enable the shutter sound.

Details:

Setting the parameter to true turns on the shutter sound, while setting it to false turns it off.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");  
mDevice.enableShutterSound(false);
```

1.2.2.7.24 enableRecordSound

Function:

```
public void enableRecordSound(boolean enable)
```

Parameters:

Enable Indicates whether to enable the record sound.

Details:

Setting the parameter to true turns on the record sound, while setting it to false turns it off.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.enableRecordSound(false);
```

1.2.2.7.25 setPictureSize

Function:

```
public void setPictureSize(android.util.Size PictureSize)
```

Parameters:

PictureSize The width and height of the picture to be taken

Details:

Used to set the size of the picture to be taken. This needs to be set before setPreviewSurface.

1.2.2.7.26 takePicture

Function:

```
public void takePicture(String fileName, ShutterCallback shutter, CamPictureCallback jpeg)
```

Parameters:

fileName The save name (including directory) of the photo

shutter Callback when the shutter sound is triggered (currently ineffective)

jpeg Callback during the photo-taking process

Details:

- ShutterCallback is defined as follows:

```
public interface ShutterCallback{
    void onShutter();
}
```

- CamPictureCallback is defined as follows:

```
public interface CamPictureCallback{
    public static final int PICTURE_TAKEN_FALL = -1; // Photo-taking failed
    public static final int PICTURE_TAKEN_SAVING = 0; // Saving in progress
    public static final int PICTURE_TAKEN_SUCCESS = 1; // Photo-taking successful
    void onPictureTaken(int status, String cameraId, String fileName);
}
```

- In onPictureTaken, status indicates the status of the photo-taking: failure, saving, and success. cameraId is the camera identifier, and fileName is the save name of the photo. If the photo-taking fails, the path will be empty. This function is called once before the photo is taken and again after completion, whether successful or not.

Examples:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
String fileName="/sdcard/DCIM/photo/"+filename;
mCameraDevice.takePicture(fileName,shutter,jpeg);
```

1.2.2.7.27 release**Function:**

```
public boolean release()
```

Details:

Unregister the callbacks related to this Camera and release the listener. This API can be called upon exiting the APK to quickly unregister callbacks and release the listener to free up resources.

Return:

If the listener is successfully removed, it returns true; otherwise, it returns false.

Examples:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
boolean isRemoveListener = mCameraDevice.release();
```

1.2.2.7.28 getCameraId**Function:**

```
public String getCameraId()
```

Returns:

Returns the ID value of the current camera (SpmCameraDevice).

Details:

Retrieves the camera ID corresponding to the current SpmCameraDevice.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
String id=mDevice.getCameraId();
```

1.2.2.7.29 startPictureSequence**Function:**

```
public void startPictureSequence(@PictureSequenceSource int source PictureConfiguration
config)
```

Parameters:

source The source for obtaining a sequence of images

config The configuration for obtaining a sequence of images

Details:

This API is for continuously obtaining images. The YUV data can be used for ADAS or other algorithmic analysis. If the image size is null, the default image size is 1280x720.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
```

```
mDevice.startPictureSequence(source , config);
```

1.2.2.7.30 stopPictureSequence

Function:

```
public void stopPictureSequence(@PictureSequenceSource int source)
```

Details:

Stop obtaining a sequence of pictures, corresponding to startPictureSequence().

Examples:

```
CameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.stopPictureSequence (source);
```

1.2.2.7.31 enableShareBuffer

Function:

```
public void enableShareBuffer(@PictureSequenceSource int source , boolean enable)
```

Parameters:

source The source for obtaining a sequence of images

enable Whether to enable FD (Face Detection) callback

Details:

This API is for continuously obtaining image data through Face Detection (FD). The YUV data can be used for ADAS or other algorithmic analysis.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.startPictureSequence(source , config);
mDevice.enableShareBuffer(source , true);
```

1.2.2.7.32 updateRequest

Function:

```
public void updateRequet()
```

Details:

This function needs to be called to take effect after updating SpmParameters.

1.2.2.7.33 getCameraId

Function:

```
public String getCameraId()
```

Details:

Retrieve the current SpmCameraDevice's ID.

1.2.2.7.34 setAeStatusCallback

Function:

```
public void setAeStatusCallback(AeStatusCallback callback)
```

Parameters:

callback The callback for AE exposure value.

Details:

- AeStatusCallback is defined as follows:

```
public interface AeStatusCallback {
    void onStatusCallback (int value);
    void onStatusCallbackArray (int[] value,int size);
}
```

Examples:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
mCameraDevice.setAeStatusCallback (
    new AeStatusCallback () {
        public void onStatusCallback (int value){
            //TODO
        }
        public onStatusCallbackArray (int[] value,int size){
            //TODO
        }
    }
);
```

1.2.2.7.35 setDropStreamFrame

Function:

```
public int setDropStreamFrame(int streamType, int streamSource, int streamPolicy)
```

Parameters:

streamType YUV 0/record 1/preview 2

streamSource @PictureSequenceSource int source ,@PreviewSource.Format int previewSource,
@RecordSource.Format int recordSource

streamPolicy Frame drop strategy

- if param == 0, do not drop frame;
- if param > 0, drop one frame each parameter;
- if param < 0, drop parameter frames each one frame.

Returns:

Returns an integer value indicating whether setting single-channel frame dropping was successful; 0 indicates success, otherwise it indicates failure.

Examples:

```
mCameraDevice.setDropStreamFrame (1,RecordSource.GENERAL_CAMERA, 1);
// For preview channel 0, record drops every other frame, effectively dropping one frame for every frame recorded.
```

1.2.2.7.36 updateVideoGPSInfo

Function:

```
public void updateVideoGPSInfo(int latitude10000,int longitude10000, @RecordSource.Format
int recordSource )
```

Parameters:

latitude10000 latitude*10000

longitude10000 longitude *10000

recordSource The source of the recording

Details:

Dynamically set GPS location information during recording and save it to the MP4 video header.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.updateVideoGPSInfo (3140000,2240000,RecordSource.GENERAL_CAMERA);
```

1.2.2.8 SpmParameters

Import Parameters:

```
import com.mediatek.smartplatform.SpmCameraDevice.SpmParameters;
```

Instantiate (create) an object:

```
SpmCameraDevice mDevice =SmartPlatformManager.get().openCameraDevice("0");
SpmParameters params=mDevice.getParameters();
```

1.2.2.8.1 enableWatermarkImage

Function:

```
public void enableWatermarkImage(boolean enable)
```

Parameters:

enable Indicates whether to add a watermark to the picture.

Details:

Setting the parameter to true means adding a watermark to the photo, while false means not adding one. This interface controls the watermark for preview, recording, and captured images.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
SpmParameters params=mDevice.getParameters();
params.enableWatermarkImage(true);
mDevice.setParameters(params);
```

1.2.2.8.2 setWatermarkImaArea

Function:

```
public void setWatermarkImgArea(int left, int top, int right, int bottom)
```

Parameters:

left The starting position on the x-axis for the image watermark area

top The starting position on the y-axis for the image watermark area
right The ending position on the x-axis for the image watermark area
bottom The ending position on the y-axis for the image watermark area

Details:

Set the area for the image watermark. The width of the image watermark must be a multiple of 32, and the height must be a multiple of 16; otherwise, it will not display correctly. The size of the watermark image should be consistent with the size of the watermark area. The starting position of the image watermark area and the text watermark area are calculated in the same way, but there are differences in width and height. Typically, setting the width and height of the image watermark area to be the same as the size of the image will suffice.

Examples:

```
params.setWatermarkImgArea(20,20,180,180);
```

1.2.2.8.3 setWatermarkImagePath

Function:

```
public void setWatermarkImagePath(String path)
```

Parameters:

path The path of the watermark image

Details:

Set the path for the watermark image; it must be a complete path.

Examples:

```
params.setWatermarkArea("/sdcard/watermark.bmp")
```

1.2.2.8.4 enableWatermarkText

Function:

```
public void enablePictureWatermarkText(boolean enable)
```

Parameters:

enable Indicates whether to add a text watermark.

Details:

Setting the parameter to true means adding a watermark, while false means not adding one. This interface controls the text watermark for preview, recording, capture, and other related interfaces.

Examples:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
SpmParameters params=mDevice.getParameters();
params.enableWatermarkText(true);
mCameraDevice.setParameters(params);
```

1.2.2.8.5 setWatermarkTextArea

Function:

```
public void setWatermarkTextArea(int left, int top, int right, int bottom)
```

Parameters:

left The starting position on the x-axis for the image watermark area

top The starting position on the y-axis for the image watermark area
right The ending position on the x-axis for the image watermark area
bottom The ending position on the y-axis for the image watermark area

Details:

Set the area for the text watermark. The four parameters mentioned above correspond to a rectangular area. The text content will be drawn within this rectangle. If the text content exceeds this area, it will be cut off and not displayed. The final position of this area on the screen is calculated as follows:

Starting position $x = \text{left} * \text{screen width} / 1000$;

Starting position $y = \text{top} * \text{screen height} / 1000$;

Rectangle width $w = (\text{right} - \text{left}) * \text{screen width} / 1000$;

Rectangle height $h = (\text{bottom} - \text{top}) * \text{screen height} / 1000$;

Examples:

```
params.setWatermarkArea(20, 20, 60, 240);
```

1.2.2.8.6 setWatermarkText

Function:

```
public void setWatermarkText(String text)
```

Parameters:

text The text to be displayed in the watermark

Details:

Set the text content to be displayed in the watermark.

Examples:

```
params.setWatermarkText("watermark");  
mCameraDevice.setParameters(params);
```

1.2.2.8.7 setWatermarkTextSize

Function:

```
public void setWatermarkTextSize(float size)
```

Parameters:

size The size of the watermark text

Details:

Set the size of the watermark text. The size of the text watermark is calculated as a proportion of the screen height, using the formula: $\text{actual text size} = \text{size} * \text{screen height} / 1000$; the unit is pixels. Therefore, the size of the text watermark is typically set to the height of the watermark area, or smaller, to ensure it is displayed properly.

Examples:

```
params.setWatermarkArea(20, 20, 60, 240); // Rectangle width is 220, height is 40  
params.setWatermarkTextSize(40); // The text watermark size should be set to 40
```

1.2.2.8.8 setWatermarkTextColor

Function:

```
public void setWatermarkTextColor(int color)
```

Parameters:

color The color of the watermark text; if not set, the default is red.

Details:

Set the color of the watermark using ARGB mode. Colors can be constructed using the `android.graphics.Color` object.

Examples:

```
params.setWatermarkTextColor(Color.RED); // The watermark text will display in red.
```

1.2.2.8.9 setWatermarkTextPosition

Function:

```
public void setWatermarkTextPosition(float x, float y)
```

Parameters:

x The horizontal offset from the starting position of the corresponding watermark area

y The vertical offset from the starting position of the corresponding watermark area

Details:

The drawn text watermark can be understood as a rectangular image. Thus, this offset is the distance from the bottom left corner of the text watermark to the starting point of the watermark area. Typically, setting the value of `y` to be the same as the height of the watermark area is sufficient.

Examples:

```
params.setWatermarkArea(20,20,60,240); //Rectangle width is 220, height is 40  
params.setWatermarkTextPosition(10,40); //The value of y is consistent with the height of  
the rectangle area
```

1.2.2.8.10 setWatermarkFontFile

Function:

```
public void setWatermarkFontFile(String file)
```

Parameters:

file The font document for the watermark

1.2.2.8.11 setWatermarkTimeMs

Function:

```
public void setWatermarkTimeMs(boolean enable)
```

Parameters:

enable Indicates whether to enable the time watermark.

1.2.2.8.12 set

Function:

```
public void set(CaptureRequest.Key<T> key, T value)
```

Parameters:

Set the request parameters for the camera, which can be based on the keys found in the native CaptureRequest, CaptureResult, and CameraCharacteristics.

Details:

This method is used to set parameters that are already supported by Android.

1.2.2.9 IpodProxy

Import IpodProxy:

```
import com.mediatek.smartplatform.IpodProxy;
```

Instantiate (create) an object:

```
IpodProxy proxy=SmartPlatformManager.get().getIpodProxy();  
or IpodProxy proxy=IpodProxy.getInstance(SmartPlatformManager.get());
```

It is recommended to use the first method to obtain the IpodProxy object, as IpodProxy also adopts the singleton pattern.

1.2.2.9.1 doShutdown

Function:

```
public int doShutdown(String reason,boolean isShutdown)
```

Parameters:

reason The reason for shutting down, which can currently be set to any string

isShutdown Indicates whether to shut down; true means to shut down, false means not to shut down.

Returns:

The execution result of the function.

Details:

- This API is used to determine whether to actually shut down after entering IPO mode, because currently pressing the power button to shut down does not actually turn off the system but enters IPO/IPOH mode. When a complete shutdown is needed, calling this API can achieve this function. If this API is called with isShutdown set to true before the IPOD process starts, you can call this API again and set the second parameter isShutdown to false to cancel the IPOD shutdown.
- It is important to note that the **reason** parameter should not contain the "=" or ";" characters.
- The return values of calling this API are as follows:
 - RESULT_SUCCESS** Indicates that the parameters of this API have been successfully passed to the IPOD process for execution. This usually occurs after shutdown and the IPOD process has already started.
 - RESULT_PARAMS_SAVED** Indicates that the parameters of this API have been successfully saved and will be passed to the IPOD process for execution once it starts.
 - RESULT_PARAMS_REPLACED** Indicates that this API has been called before, and the parameters passed down this time override the previous ones, meaning the last parameters passed down will be executed by IPOD.

RESULT_FAIL Indicates that the call to this API failed. There are many reasons for failure, which depend on the specific situation.

RESULT_EXCEPTION Indicates that an exception occurred while calling this API, usually because the Smartplatform service does not exist, meaning the SmartPlatformServer process has died.

Examples:

```
IpodProxy proxy= SmartPlatformManager.get().getIpodProxy();
proxy.doShutdown("test",true); //After entering IPO, proceed with shutdown.
```

1.2.2.9.2 setRebootControl

Function:

```
public int setRebootControl(int flag)
```

Parameters:

flag The flag to enable the scheduled reboot feature, with values of 0 or 1

Returns:

The execution result of the function.

Details:

- Used to enable the scheduled reboot feature. Passing a flag parameter of 1 means to enable, and a parameter of 0 means to disable. If setRebootTimeSlot is not called to set the reboot time, the last set time will be used as the reboot time.
- The return values of calling this API are as follows:
 - RESULT_SUCCESS** Indicates that the parameters of this API have been successfully passed to the IPOD process for execution. This usually occurs after shutdown and the IPOD process has already started.
 - RESULT_PARAMS_SAVED** Indicates that the parameters of this API have been successfully saved and will be passed to the IPOD process for execution once it starts.
 - RESULT_PARAMS_REPLACED** Indicates that this API has been called before, and the parameters passed down this time override the previous ones, meaning the last parameters passed down will be executed by IPOD.
 - RESULT_FAIL** Indicates that the call to this API failed. There are many reasons for failure, which depend on the specific situation.
 - RESULT_EXCEPTION** Indicates that an exception occurred while calling this API, usually because the Carcorder service does not exist, meaning the SmartPlatformServer process has died.

Examples:

```
IpodProxy proxy= SmartPlatformManager.get().getIpodProxy();
proxy.setRebootControl(1); //Enable the scheduled reboot feature.
```

1.2.2.9.3 setRebootTimeSlot

Function:

```
public int setRebootTimeSlot(int slotTime)
```

Parameters:

slotTime The time point for reboot, with values ranging from 0 to 23

Returns:

The execution result of the function.

Details:

- This API is used to set the time for reboot. The parameter is the time point for reboot, ranging from 0 to 23. To make the current setting effective, setRebootControl must be called to enable the scheduled reboot feature.
- The return values of calling this API are as follows:
 - RESULT_SUCCESS** Indicates that the parameters of this API have been successfully passed to the IPOD process for execution. This usually occurs after shutdown and the IPOD process has already started.
 - RESULT_PARAMS_SAVED** Indicates that the parameters of this API have been successfully saved and will be passed to the IPOD process for execution once it starts.
 - RESULT_PARAMS_REPLACED** Indicates that this API has been called before, and the parameters passed down this time override the previous ones, meaning the last parameters passed down will be executed by IPOD.
 - RESULT_FAIL** Indicates that the call to this API failed. There are many reasons for failure, which depend on the specific situation.
 - RESULT_EXCEPTION** Indicates that an exception occurred while calling this API, usually because the Smartplatform service does not exist, meaning the SmartPlatformServer process has died.

Examples:

```
IpodProxy proxy= SmartPlatformManager.get().getIpodProxy();
proxy.setRebootControl(1); // Enable the scheduled reboot feature.
proxy.setRebootTimeSlot(5); // Set to reboot at 5 AM.
```

1.2.2.9.4 exitIpod**Function:**

```
public int exitIpod(int reason)
```

Parameters:

reason The reason for exiting IPOD mode

Returns:

The execution result of the function.

Details:

- In IPO mode, calling this API has the same function as long-pressing the power button to start the device. It is effective only after the IPOD process has started.
- The return values of calling this API are as follows:
 - RESULT_SUCCESS** Indicates that the parameters of this API have been successfully passed to the IPOD process for execution. This usually occurs after shutdown and the IPOD process has already started.
 - RESULT_PARAMS_SAVED** Indicates that the parameters of this API have been successfully saved and will be passed to the IPOD process for execution once it starts.
 - RESULT_PARAMS_REPLACED** Indicates that this API has been called before, and the parameters passed down this time override the previous ones, meaning the last parameters passed down will be executed by IPOD.
 - RESULT_FAIL** Indicates that the call to this API failed. There are many reasons for failure, which depend on the specific situation.
 - RESULT_EXCEPTION** Indicates that an exception occurred while calling this API, usually because the Carcorder service does not exist, meaning the SmartPlatformServer process has died.

Examples:

```
IpodProxy proxy= SmartPlatformManager.get().getIpodProxy();
proxy.exitIpod(0); // Exit IPOD and then start the device.
```

1.2.2.10 CarEventProxy

Import CarEventProxy:

```
import com.mediatek.smartplatform.CarEventProxy;
```

Instantiate (create) an object:

```
CarEventProxy proxy=SmartPlatformManager.get().getCarEventProxy();  
or CarEventProxy proxy= CarEventProxy.getInstance(SmartPlatformManager.get());
```

It is recommended to use the first method to obtain the CarEventProxy object, as CarEventProxy also utilizes the singleton pattern. Due to hardware limitations, currently only ACC on/off behavior is supported. Other behaviors need to be added according to the hardware specifications.

1.2.2.10.1 addEngineChangedCallback

Function:

```
public void addEngineChangedCallback(EngineChangedCallback callback)
```

Parameters:

callback The callback when the car engine state changes, corresponding to ACC on/off. If the same callback object is registered multiple times, only one instance will be saved.

Details:

- EngineChangedCallback is defined as follows:

```
public interface EngineChangedCallback{  
    public static final int CAR_ENGINE_FLAMEOUT =0;  
    public static final int CAR_ENGINE_WORKING = 1;  
    void onEngineChanged(int status);  
}
```

- This callback is triggered when the car engine state changes. The status in onEngineChanged indicates the engine's state. When the value is CAR_ENGINE_FLAMEOUT, it means the engine is off. When the value is CAR_ENGINE_WORKING, it means the engine is running.
- These two events, CAR_ENGINE_WORKING/CAR_ENGINE_FLAMEOUT, correspond to the ACC on/off events.

Examples:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();  
mCarEventProxy.addEngineChangedCallback(  
    new EngineChangedCallback(){  
        public void onEngineChanged (int status){  
            //TODO  
        }  
    }  
);
```

1.2.2.10.2 removeEngineChangedCallback

Function:

```
public void removeEngineChangedCallback(EngineChangedCallback callback)
```

Parameters:

callback The callback when the car engine state changes

Details:

Remove the registered callback object.

Examples:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();  
mCarEventProxy.removeEngineChangedCallback(callback);
```

1.2.2.10.3 queryCarEngineState

Function:

```
public int queryCarEngineState()
```

Returns:

The current working state of the engine.

Details:

Currently, three states are defined:

- CAR_STATE_UNKNOWN indicates the state is unknown, which generally occurs when an API call results in an exception.
- CAR_ENGINE_FLAMEOUT indicates the engine is off.
- CAR_ENGINE_WORKING indicates the engine is running. The engine state saved by the Smartplatform Service is directly returned.

Examples:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();  
int state= mCarEventProxy.queryCarEngineState();
```

1.2.2.10.4 setDefaultAccOffBehavior

Function:

```
public void setDefaultAccOffBehavior(boolean enable)
```

Parameters:

enable Indicates whether to enable the default ACC behavior.

Details:

- The default ACC behavior is that when the underlying Driver detects ACC ignition or shutdown, it sends this event to the app in the form of a broadcast. The app only needs to register the corresponding broadcast. The ignition event broadcast is `android.intent.action.ACTION_POWER_CONNECTED`, and the shutdown event broadcast is `android.intent.action.ACTION_POWER_DISCONNECTED`.
- If the app wants to receive ACC ignition or shutdown events through callbacks, set the parameter value to false and register the corresponding callback through `addEngineChangedCallback`.

Examples:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();  
mCarEventProxy.addEngineChangedCallback(  
    new EngineChangedCallback(){  
        public void onEngineChanged (int status){  
            //TODO  
        }  
    }  
);  
mCarEventProxy.setDefaultAccOffBehavior(false);
```

1.2.2.10.5 addCarReverseCallback

Function:

```
public void addCarReverseCallback(CarReverseCallback callback)
```

Parameters:

callback The callback when the car's direction of movement changes. If the same callback object is registered multiple times, only one instance will be saved.

Details:

- CarReverseCallback is defined as follows:

```
public interface CarReverseCallback {
    public static final int CAR_STATUS_NORMAL = 0;
    public static final int CAR_STATUS_LEFT = 1;
    public static final int CAR_STATUS_RIGHT = 2;
    public static final int CAR_STATUS_REVERSE = 3;

    public static final int CAR_STATUS_SOURCE_AVM = 1;
    public static final int CAR_STATUS_SOURCE_GPIO = 2;

    void onReverse(int status, int source);
}
```

- This callback is triggered when the car transitions from one state (left turn/right turn/reverse/forward) to a different state. There are two sources that can trigger this callback: GPIO or AVM. Both triggering methods require hardware support.

Examples:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();
mCarEventProxy.addCarReverseCallback(
    new CarReverseCallback() {
        public void onReverse(int status, int source) {
            //TODO
        }
    }
);
```

1.2.2.10.6 removeCarReverseCallback

Function:

```
public void removeCarReverseCallback(CarReverseCallback callback)
```

Parameters:

callback The callback when the car's motion state changes

Details:

Remove the registered callback object.

Examples:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();
mCarEventProxy.removeCarReverseCallback(callback);
```

1.2.2.11 CollisionProxy

Import CollisionProxy:

```
import com.mediatek.smartplatform.CollisionProxy;
```

Instantiate (create) an object:

```
CollisionProxy proxy=SmartPlatformManager.get().getCollisionProxy();
```

```
Or CollisionProxy proxy= CollisionProxy.getInstance(SmartPlatformManager.get());
```

It is recommended to use the first method to obtain the CollisionProxy object, as CollisionProxy also adopts the singleton pattern.

1.2.2.11.1 addCollisionCallback

Function:

```
public void addCollisionCallback(CollisionCallback callback)
```

Parameters:

callback The callback when the platform detects a collision

Details:

- CollisionCallback is defined as follows:

```
public interface CollisionCallback{
    public static final int COLLISION_UNRELIABLE =0;
    public static final int COLLISION_LOW=1;
    public static final int COLLISION_MEDIUM=2;
    public static final int COLLISION_HIGH=3;
    void onCollision(int collision,int status);
}
```

- This callback is triggered when the platform detects a collision. The collision in onCollision actually refers to the level (sensitivity) of the collision, currently divided into three levels: COLLISION_LOW, COLLISION_MEDIUM, COLLISION_HIGH. The status is a reserved parameter, generally 0. If the collision detection algorithm is modified, then this value will be equal to the (*result) value in collision_detect().
- The same object can only be registered once. Even if the object is added multiple times, ultimately only one callback object will be saved. Similarly, you only need to call removeCollisionCallback once to completely remove the callback.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
mSmartPlatformManager.addCollisionCallback(
    new CollisionCallback() {
        public void onCollision(int collision,int status){
            //TODO
        }
    }
);
```

1.2.2.11.2 removeCollisionCallback

Function:

```
public void removeCollisionCallback (CollisionCallback callback)
```

Parameters:

`callback` The callback when the platform detects a collision

Details:

Remove the registered callback object. It's worth noting that if the same callback object has been registered multiple times, calling this function once will completely remove it.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.removeCollisionCallback(callback);
```

1.2.2.11.3 setSuspendCollision

Function:

```
public void setSuspendCollision(boolean fgEnabled)
```

Parameters:

`fgEnabled` Indicates whether to enable the Gsensor collision detection feature.

Details:

- This API can be used to enable or disable the Gsensor collision detection feature. A parameter of true means to enable, and false means to disable. This API corresponds to the collision boot feature. By default, this feature is enabled, and you can use `getSuspendCollision` to check whether this feature is enabled.
- Whether it is a normal boot or a collision boot can be distinguished by the broadcast sent out at boot time.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
// Enable the Gsensor collision detection feature, so that the device will boot up if a  
collision occurs even in IPO status  
mSmartPlatformManager.setSuspendCollision(true);
```

1.2.2.11.4 getSuspendCollision

Function:

```
public boolean getSuspendCollision()
```

Returns:

A boolean value indicating whether the Gsensor Driver collision detection is enabled.

Details:

If the return value is true, it means it is enabled; if false, it means it is disabled.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
boolean isEnabled=mSmartPlatformManager.getSuspendCollision();
```

1.2.2.11.5 setNormalCollision

Function:

```
public void setNormalCollision(boolean isEnabled)
```

Parameters:

`isEnabled` Indicates whether to enable the Carcorder collision detection feature.

Details:

The Carcorder collision detection can be understood as a feature module within the Carcorder process. This interface can control the on/off switch of this feature. Only when this feature is enabled, and a collision occurs, will the `CollisionCallback` receive a callback. Carcorder collision detection and Gsensor collision detection belong to different modules. Disabling the Gsensor collision detection feature will not affect the Carcorder collision detection feature, and vice versa. Therefore, these two features do not affect each other. By default, this feature is enabled, and if it was set previously, the previous settings will be used.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
mSmartPlatformManager.addCollisionCallback(
    new CollisionCallback {
        public void onCollision(int collision,int status){
        }
    }
);
mSmartPlatformManager.setNormalCollision(true);
```

1.2.2.11.6 `getNormalCollision`

Function:

```
public boolean getNormalCollision()
```

Returns:

A boolean value indicating whether the Carcorder collision detection feature is enabled.

Details:

If the return value is true, it means it is enabled; if false, it means it is disabled.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
boolean isEnabled=mSmartPlatformManager.getNormalCollision();
```

1.2.2.11.7 `setSuspendCollisionSensity`

Function:

```
public void setSuspendCollisionSensity(int level)
```

Parameters:

`level` Sensitivity of the collision

Details:

The parameter level actually refers to sensitivity, meaning that if the change in the Gsensor exceeds a certain threshold, it is considered a collision. This threshold corresponds to the level, which is currently defined as `COLLISION_LOW`, `COLLISION_MEDIUM`, and `COLLISION_HIGH`. By default, if not set, it is `COLLISION_MEDIUM`.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
mSmartPlatformManager.setSuspendCollisionSensity(CollisionCallback.COLLISION_HIGH);
```

1.2.2.11.8 getSuspendCollisionSensity

Function:

```
public int getSuspendCollisionSensity()
```

Returns:

Returns an integer value indicating the sensitivity with which the Driver will detect a collision.

Details:

Currently three levels are defined: `COLLISION_LOW`, `COLLISION_MEDIUM`, and `COLLISION_HIGH`, with `COLLISION_HIGH` indicating a higher sensitivity to detecting collisions. For example, setting it to `COLLISION_HIGH` might result in detecting a collision from a light touch.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
int level=mSmartPlatformManager.getDriverCollisionSensity();
```

1.2.2.11.9 setNormalCollisionSensity

Function:

```
public void setNormalCollisionSensity(int level)
```

Parameters:

level Sensitivity of the collision

Details:

The parameter level refers to sensitivity, meaning that if the Gsensor's change exceeds a certain threshold, it is considered a collision. This threshold corresponds to the level, currently defined as `COLLISION_LOW`, `COLLISION_MEDIUM`, and `COLLISION_HIGH`. Although this function is similar to `setSuspendCollisionSensity()`, they belong to different modules and do not affect each other. If you need to customize the collision threshold, you can do so through `setCollisionThreshold`. `setCollisionThreshold()` can completely replace this interface because the set sensitivity will be converted to the corresponding threshold value, which has the same effect as directly modifying the threshold value.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
mSmartPlatformManager.setNormalCollisionSensity(CollisionCallback.COLLISION_HIGH);
```

1.2.2.11.10 getNormalCollisionSensity

Function:

```
public int getNormalCollisionSensity()
```

Returns:

Returns an integer value indicating the sensitivity with which the Carcorder will detect a collision.

Details:

Currently three levels are defined: `COLLISION_LOW`, `COLLISION_MEDIUM`, and `COLLISION_HIGH`, with `COLLISION_HIGH` indicating a higher sensitivity to detecting collisions. If `setCollisionThreshold` is used to set a defined collision threshold, then the return value of this function will be the level value set by `setCollisionThreshold`.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
```

```
int level=mSmartPlatformManager.getNormalCollisionSensity();
```

1.2.2.11.11 setCollisionThreshold

Function:

```
public int setCollisionThreshold(float x,float y,float z,int level)
```

Parameters:

- x Threshold for Gsensor acceleration in the X-axis direction
- y Threshold for Gsensor acceleration in the Y-axis direction
- z Threshold for Gsensor acceleration in the Z-axis direction
- level The level represented by this threshold

Returns:

Returns an integer value indicating whether the parameter setting was successful, 0 for success, otherwise failure.

Details:

- This API allows for customizing the collision thresholds, which are the Gsensor acceleration thresholds in the X, Y, and Z directions. The previous collision detection algorithm considered a collision detected if the Gsensor's acceleration in any direction exceeded the corresponding threshold. The current algorithm considers a collision detected if the Gsensor's acceleration change in any direction exceeds the set threshold.
- The fourth parameter, `level`, is not limited and can be any integer value. This parameter acts like an identifier, so when a collision is detected, the first parameter in `CollisionCallback.onCollision()` will be this set value. This value can also be obtained through `getNormalCollisionSensity`.
- This function serves the same purpose as `setNormalCollisionSensity()`, so there is no need to call both interfaces. If both are called, only the parameters set by the last called interface will be used. It is recommended to use this interface.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
mSmartPlatformManager.setCollisionThreshold(5f,5f,5f,2);
```

1.2.2.11.12 getCollisionThreshold

Function:

```
public float[] getCollisionThreshold()
```

Returns:

Returns the set threshold values for Gsensor acceleration in the X, Y, and Z-axis directions.

Details:

This API allows you to retrieve the Gsensor acceleration thresholds in the X, Y, and Z-axis directions that were set by `setCollisionThreshold`. If the return value is null, it indicates that there was an exception while retrieving the thresholds.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
float[] f=mSmartPlatformManager.getCollisionThreshold();
```

1.2.2.11.13 setGsensorEventRate

Function:

```
public int setGsensorEventRate(int delayMs)
```

Parameters:

delayMs The interval for reporting Gsensor data, in milliseconds

Returns:

Returns an integer value indicating whether the parameter setting was successful, 0 for success, otherwise failure.

Details:

- This API allows you to set the interval for reporting Gsensor data, in milliseconds. For example, if set successfully to 10, it means that Gsensor data will be received every 10 milliseconds, which translates to a frequency of 100Hz.
- This interface simply calls the setting interface of the sensorservice, which then sets the value to the driver. The value passed in and the actual value set to the driver may not be equal because there are range limitations for this value.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.setGsensorEventRate(10);
```

1.2.2.11.14 getGsensorEventRate

Function:

```
public int getGsensorEventRate()
```

Returns:

Returns the set interval for reporting Gsensor data.

Details:

This interface simply retrieves the value passed to the last call of setGsensorEventRate(), not the actual value used by the driver. Currently, there is no way to retrieve the actual value from the driver.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
int rate=mSmartPlatformManager.getGsensorEventRate();
```

1.2.2.11.15 setIPOStandby

Function:

```
public void setIPOStandby(Boolean on)
```

Parameters:

On True to enter IPO standby mode; false to exit IPO standby mode.

Details:

NATIVE Collision will choose different flows based on whether it is in IPO standby mode or not, in order to achieve power saving.

Examples:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.setIPOStandby ();
```

1.3 Configuration/Customization Guideline

Codebase path: vendor\mediatek\proprietary\frameworks\base\smartplatform\sdk

Apk demo path: vendor\mediatek\proprietary\packages\apps\CarcorderMcamDemo

1.3.1 Main Features Introduction

1.3.1.1 Preview

Start preview:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");  
SurfaceView surfaceView=findViewById(R.id.surface_view);  
SurfaceHolder holder=surfaceView.getHolder();  
mDevice.setPreviewSurface(holder.getSurface(), PreviewSource.GENERAL_CAMERA);  
mDevice.startPreview();
```

Stop preview:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");  
mDevice.stopPreview();  
mDevice.setPreviewSurface(null, PreviewSource.GENERAL_CAMERA);
```

1.3.1.2 TakePicture

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");  
String fileName="/sdcard/DCIM/photo/"+filename;  
mCameraDevice.takePicture(fileName, shutter, jpeg);
```

1.3.1.3 Record

Start record:

```
RecordConfiguration recordConfig= RecordConfiguration.get(recordSource);  
recordConfig.setVideoCallback(mVideoCallback);  
mCameraDevice.startRecord(recordSource, recordConfig);
```

Stop record:

```
SpmCameraDevice Device = SmartPlatformManager.get().openCameraDevice("0");  
mDevice.stopRecord();
```

Note:

- Recording involves various features such as sub-stream, main stream, event marking protection, etc., which will be detailed in the DVR documentation.

1.3.1.4 YUV Callback

```
SpmCameraDevice.ImageDataCallback mSequenceCallback = new SpmCameraDevice.ImageDataCallback() {
    Public void onImageAvailable(ImageDataCallbackInfo imageDataCallbackInfo) {
    }
}

SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
PictureConfiguration picConfig = PictureConfiguration.get(PictureSequenceSource.GENERAL_CAMERA);
picConfig.mImageDataCallback = mSequenceCallback;
mDevice.startPictureSequence(source ,picConfig);
```

Note:

- The imageDataCallback returns data; if a File Descriptor (FD) callback is needed, it should be obtained through enableShareBuffer when calling startPictureSequence.

1.3.2 Special Notes on Collision Detection

1.3.2.1 Starting a Recording APK upon Collision

Requirement Description:

When the device is powered on, after the Demo APK exits, it is not possible to receive collision notifications sent by the Carcorder Service through the registered callback. Now, to receive collision events after the APK exits (when it is not running), MediaTek uses a broadcast to notify the APK of the collision event.

Sample Code:

- Implement a class that extends BroadcastReceiver

```
class CollisionBroadcastReceiver extends BroadcastReceiver{
@Override
public void onReceive(Context context, Intent intent)
{
}
}
```

- The broadcast should be registered statically in the Manifest.xml

```
<receiver android:name=".CollisionBroadcastReceiver">
    <intent-filter>
        <action
            android:name="android.intent.action.GSENSOR_COLLISION"/>
    </intent-filter>
</receiver>
```

1.3.2.2 Open and Customizable Collision Detection Algorithm

Code Path:

alps/frameworks/native/libs/smartplatform/libsmartplatformcollision/*

File Composition:

SmartPlatformCollision.h SmartplatformCollision.cpp Android.mk

Interface Definition:

The definitions of the following two functions cannot be changed, including the function names, parameters, and return values.

void collision_init();

Initialization function is called once when the collide starts running and after libsmartplatformcollision.so is loaded, for variable initialization.

void collision_detect(AsensorEvent value,const float* threshod, int size, int level, int* result);

Parameter Description:

AsensorEvent value: A structure encapsulating sensor data.

const float*threshold: An array of thresholds for x, y, z axes, consistent with the values set by setCollisionThreshold().

int size: Specifies the length of the threshold array, currently defined as 3.

int level: The level of collision, also the value set before.

int* result: An additional return value that the app can obtain through CollisionCallback.onCollision(int collision, int status), where collision is equal to level, and status is equal to *result.

Return Value Description:

If it returns true, it is considered that a collision has been detected, and the app will be notified through a callback. If the app does not register a callback, a broadcast will be sent out. If it returns false, it is considered that no collision has been detected, and neither a callback nor a broadcast will be sent.

Other Notes:

sensorservice will call the collision_detect function each time it sends data. The function setNormalCollision() can be used to enable or disable this feature. If this feature is disabled, collision_detect will not be called even if sensor data is sent.

Reference Example:

Please refer to the implementation of SmartplatformCollision.h and SmartPlatformCollision.cpp.

1.4 Frequently Asked Questions/Troubleshooting

1.4.1 Common SDK Questions

1.4.1.1 Incorrect Order of Calling setpreviewSurface Causing the Underlying Buffer to Get Stuck

If setpreviewSurface null is invoked before stop preview, the following printout may occur:

BufferQueueProducer: [ImageReader-1280x720f32315659m5-15749-52] (this:0x7a989dd000,id:52,api:4,p:544,c:-1) queueBuffer: BufferQueue has been abandoned

This error occurs because the surface is destroyed before invoking stop preview, causing the underlying buffer to get stuck.

1.4.1.2 Taking a Picture (takePicture) in a Separate Thread

Some SD cards are slow to save pictures, so `takePicture` can be run in a separate thread to ensure it does not freeze the APK.

```
new Thread(() -> {  
    mDevice.takePicture(path, mShutterCallback, pictureCallback);  
}).start();
```

1.4.1.3 Do not Perform Time-Consuming Actions in Callbacks

Considering synchronization and efficiency issues, callbacks in the SDK are processed directly in the binder thread without starting a new thread. Therefore, it is best for the app to start a new thread to handle the callback after receiving it. If a new thread is not started, do not continue to call SDK interfaces in the callback (as it may cause a deadlock), and do not perform time-consuming actions in the callback, as this can easily cause the APK to ANR.

1.4.2 Methods to Toggle Debug Logs

1.4.2.1 Method to Enable SpmCameraDeviceImpl Debug Log

```
adb shell setprop persist.vendor.log.spm sdk true;  
adb reboot
```

1.4.2.2 Method to Disable the Pre-Set Capture Surface

```
adb shell setprop persist.vendor.spm.presetpic false  
adb reboot
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.